PROJECT REPORT

Design and Verification of a 32-bit Single-Cycle RISC-V (RV32I) Processor in Verilog

Behavioural simulation and functional verification

Abstract

This report presents the design, implementation and functional verification of a 32-bit single-cycle processor based on the RISC-V RV32I base integer instruction set. The processor is described in synthesizable Verilog and is organised into an Arithmetic Logic Unit (ALU), a 32 x 32-bit register file, an immediate generator, a datapath with instruction decode and control logic, a 64-word data memory and a top-level CPU wrapper. Every instruction completes in exactly one clock cycle.

The design supports register-register and register-immediate arithmetic and logic instructions, word load and store (LW/SW), six conditional branches, JAL, JALR, LUI and AUIPC. A self-checking testbench loads a 17-instruction test program into an instruction ROM, runs the processor for 21 clock cycles and compares the final register and memory contents with expected values. All 15 checks (14 register checks and 1 memory check) passed. Waveforms of the program counter, register file ports, ALU, memory interface, branch logic and write-back path confirm correct behaviour cycle by cycle.

Keywords: *RISC-V, RV32I, single-cycle processor, Verilog HDL, ALU, register file, testbench, simulation.*

1. Introduction

RISC-V is an open, royalty-free instruction set architecture (ISA) that has become widely used in teaching, research and industry. Its regular instruction formats and small base set (RV32I) make it an excellent platform for learning processor design. In a single-cycle implementation, fetch, decode, register read, execute, memory access and write-back all happen within one clock period, so the cycles-per-instruction (CPI) is exactly one. This keeps the control logic simple and makes the processor easy to understand and verify.

This project implements such a processor in Verilog HDL and verifies it using a directed testbench. The instruction memory is modelled as a ROM inside the testbench and the data memory is a separate module, which follows the Harvard-style separation commonly used for single-cycle designs.

1.1 Objectives

- Design a 32-bit single-cycle processor that executes a useful subset of the RV32I instruction set.
- Implement the ALU, register file, immediate generator, control logic and data memory as separate reusable Verilog modules.
- Write a self-checking testbench with a directed assembly test program covering arithmetic, logic, memory, branch and jump instructions.
- Verify correctness using console output (PASS/FAIL) and waveform analysis.

2. System Architecture

The processor follows the classic single-cycle datapath. The program counter (PC) addresses the instruction ROM. The fetched instruction is decoded to select the ALU operation, the second ALU operand, the write-back source and the next PC. Register operands are read combinationally from the register file, the ALU computes a result (or a memory address), and the result is written back to the register file at the next rising clock edge. Fig. 1 shows the overall block diagram.

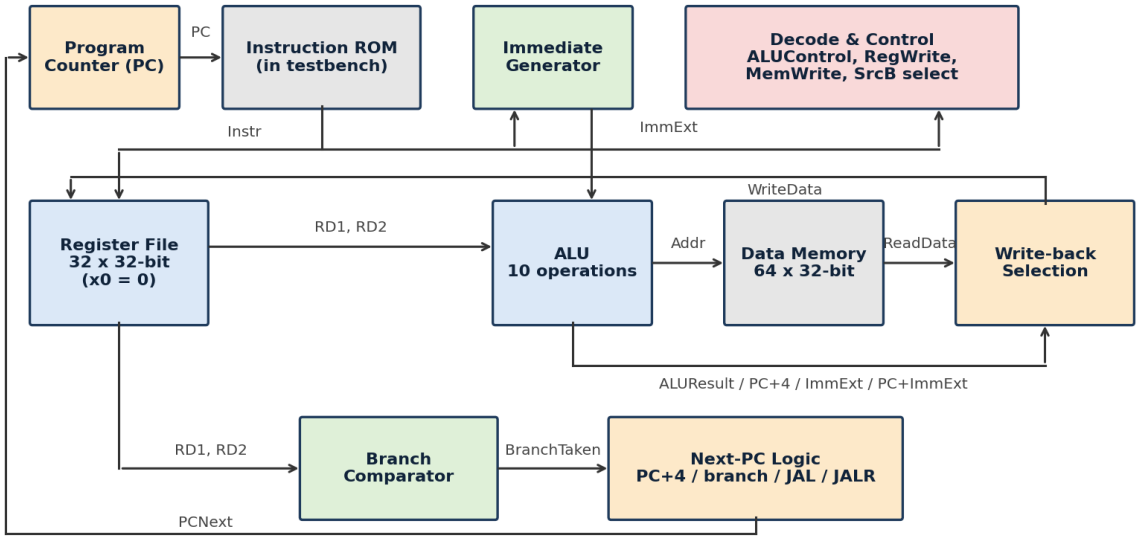


Fig. 1 Block diagram of the single-cycle RISC-V processor

2.1 Module Summary

Table 1 Verilog modules in the design

Module	Function
`alu`	32-bit combinational ALU with 10 operations and a Zero flag.

Module	Function
register_file	32 x 32-bit register file; two asynchronous read ports, one synchronous write port; x0 hard-wired to zero.
extend_unit	Immediate generator for I, S, B, U and J formats with sign extension.
datapath	Instruction decoding, ALU control, ALU source select, branch decision, write-back select, next-PC logic and the PC register.
riscv_cpu	Top-level wrapper exposing PC, Instr, memory interface and Result.
data_mem	64-word x 32-bit data memory; synchronous word-aligned write, combinational read.

3. Module Description

3.1 Arithmetic Logic Unit (alu)

The ALU takes two 32-bit operands A and B and a 4-bit ALUControl signal. It is purely combinational and implemented with a case statement. The Zero output is asserted whenever the result equals zero. Shift amounts use only the lower five bits of operand B (B[4:0]), as required by RV32I.

Table 2 ALU control encoding

ALUControl	Operation	Description
0000	ADD	A + B (also used for address calculation and JALR target)
0001	SUB	A - B (SUB instruction and branch compare)
0010	AND	Bitwise AND
0011	OR	Bitwise OR
0100	SLL	Logical shift left by B[4:0]
0101	SLT	Signed set-less-than, result is 1 or 0
0110	SLTU	Unsigned set-less-than, result is 1 or 0
0111	XOR	Bitwise XOR
1000	SRL	Logical shift right by B[4:0]
1001	SRA	Arithmetic shift right by B[4:0] (sign preserved)

3.2 Register File (register_file)

The register file contains 32 registers of 32 bits each. Two read ports (A1, A2) are asynchronous, so operands are available in the same cycle as the instruction. The single write port (A3, WD3) writes on the rising clock edge when the write enable WE3 is high. Writes to register x0 are blocked and reads of x0 always return zero, which satisfies the RISC-V requirement that x0 is constant zero. An asynchronous reset clears all registers.

3.3 Immediate Generator (extend_unit)

The immediate generator decodes the opcode and rebuilds the sign-extended 32-bit immediate for the five RISC-V immediate formats: I-type (ADDI, loads, JALR), S-type (stores), B-type (branches, with implicit zero in bit 0), U-type (LUI, AUIPC, upper 20 bits) and J-type (JAL, with implicit zero in bit 0). For unsupported opcodes the output is zero.

3.4 Datapath and Control (datapath)

The datapath instantiates the register file, immediate generator and ALU and contains all control logic in combinational always blocks. The main functions are:

- **Field extraction:** opcode, rd, funct3, rs1, rs2 and funct7 are taken directly from the instruction bits.
- **ALU source select:** SrcB is the immediate for LOAD, STORE, OP-IMM and JALR, and RD2 otherwise.
- **ALU operation select:** for R-type and I-type ALU instructions funct3 (and funct7 for SUB, SRA/SRAI) selects the operation; branches use SUB; all other instructions use ADD for address calculation.
- **Branch decision:** a dedicated comparator evaluates BEQ, BNE, BLT, BGE, BLTU and BGEU on RD1 and RD2 and drives BranchTaken.
- **Write-back select:** ALU result for R/I-type, ReadData for loads, ImmExt for LUI, PC + ImmExt for AUIPC and PC + 4 for JAL/JALR. RegWrite is low for stores and branches.
- **Next-PC logic:** PC + 4 by default; PC + ImmExt for a taken branch or JAL; (RD1 + imm) with bit 0 cleared for JALR.

Memory write signals are derived directly: MemWrite is high for the STORE opcode, Mem_WrAddr is the ALU result and Mem_WrData is RD2.

3.5 Data Memory (data_mem)

The data memory has 64 words of 32 bits (256 bytes). It is initialised to zero. A write occurs on the rising clock edge when WE is high and the address is word aligned (address bits [1:0] = 00); the word index is taken from address bits [7:2]. The read is combinational, so a load instruction can read and write back in the same cycle. The same ALU-generated address (Mem_WrAddr) is used for both reads and writes.

3.6 Top-Level CPU (riscv_cpu)

The top level simply wraps the datapath and exposes the PC, instruction input, data-memory interface and a debug Result output (the value being written back). The instruction ROM and data memory are connected externally by the testbench.

4. Supported Instructions

Table 3 Instructions implemented by the processor

Type	Instructions	Opcode	Operation
R-type	ADD, SUB, AND, OR, XOR, SLL, SLT, SLTU, SRL, SRA	0110011	rd = rs1 op rs2
I-type ALU	ADDI, ANDI, ORI, XORI, SLLI, SLTI, SLTIU, SRLI, SRAI	0010011	rd = rs1 op imm
Load	LW	0000011	rd = Mem[rs1 + imm]
Store	SW	0100011	Mem[rs1 + imm] = rs2
Branch	BEQ, BNE, BLT, BGE, BLTU, BGEU	1100011	PC = PC + imm if condition true
JAL	JAL	1101111	rd = PC + 4; PC = PC + imm
JALR	JALR	1100111	rd = PC + 4; PC = (rs1 + imm) & ~1
U-type	LUI, AUIPC	0110111 / 0010111	rd = imm / rd = PC + imm

5. Verification Methodology

5.1 Testbench Environment

The testbench (tb_riscv) instantiates the CPU and the data memory and models the instruction memory as a 32-word ROM indexed by PC[6:2] (the PC is a byte address and each instruction occupies four bytes). A 10 ns clock (period) is generated, reset is held high for two clock cycles and then released at 20 ns. After reset the testbench prints the PC, instruction, write-back result and MemWrite for 21 clock cycles, and then calls two checking tasks:

- **check_reg(n, expected):** compares register xn inside the register file with the expected value and prints PASS or FAIL.
- **check_mem(index, expected):** compares a data-memory word with the expected value.

An error counter is incremented on every mismatch, and the testbench finally prints either ALL TESTS PASSED or the number of errors. A VCD file is also dumped for waveform viewing.

5.2 Test Program

The directed test program contains 17 instructions. Unused ROM locations are filled with NOP (ADDI x0, x0, 0). The program covers register-immediate and register-register arithmetic and logic, a store followed by a load from the same address, a taken branch that must skip one instruction, and a JAL that must skip one instruction and write the return address.

Table 4 Test program loaded into the instruction ROM

PC	Machine code	Assembly	Expected effect
0	00500093	ADDI x1, x0, 5	x1 = 5
4	00A00113	ADDI x2, x0, 10	x2 = 10
8	002081B3	ADD x3, x1, x2	x3 = 15

PC	Machine code	Assembly	Expected effect
12	40110233	SUB x4, x2, x1	x4 = 5
16	0020F2B3	AND x5, x1, x2	x5 = 0
20	0020E333	OR x6, x1, x2	x6 = 15
24	0020C3B3	XOR x7, x1, x2	x7 = 15
28	0020A433	SLT x8, x1, x2	x8 = 1
32	00302023	SW x3, 0(x0)	Mem[0] = 15
36	00002483	LW x9, 0(x0)	x9 = 15
40	00100513	ADDI x10, x0, 1	x10 = 1
44	00A50463	BEQ x10, x10, +8	Branch taken, PC becomes 52
48	06300593	ADDI x11, x0, 99	Must be skipped
52	00700593	ADDI x11, x0, 7	x11 = 7
56	0080066F	JAL x12, +8	x12 = 60, PC becomes 64
60	05800693	ADDI x13, x0, 88	Must be skipped
64	00900693	ADDI x13, x0, 9	x13 = 9

6. Simulation Results and Analysis

The design was simulated with the testbench above and the waveforms were viewed in the EPWave viewer. The clock period is 10 ns and the plots cover 0 to 220 ns. In the waveforms the instruction word is displayed in decimal radix (for example 5243027 equals 0x00500093, the first ADDI). Reset is asserted initially and goes low at 20 ns.

6.1 Program Counter and Next-PC Behaviour

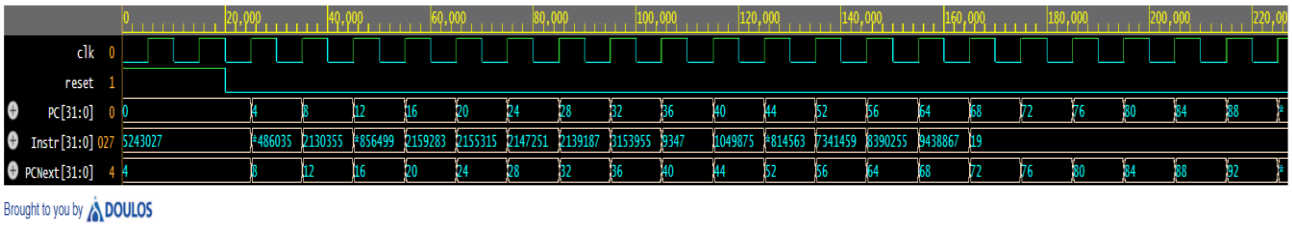


Fig. 2 Waveform of clk, reset, PC, Instr and PCNext

Fig. 2 shows that the PC starts at 0 and increases by 4 every clock cycle (0, 4, 8, ... 44), meaning one instruction is executed per cycle. At PC = 44 the BEQ instruction is taken: PCNext is 52 instead of 48, so the instruction at address 48 is skipped. At PC = 56 the JAL instruction makes PCNext equal 64, skipping the instruction at address 60. After the last program instruction at PC = 64 the processor executes NOPs and the PC continues in steps of 4 (68, 72, 76, ...).

6.2 Register Read, ALU and Write-back

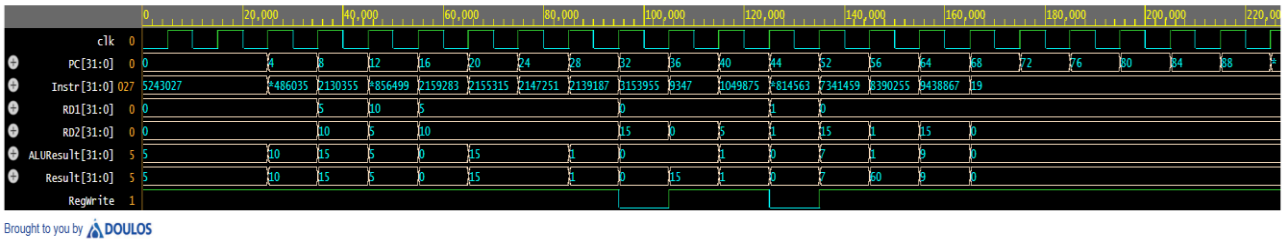


Fig. 3 Waveform of RD1, RD2, ALUResult, Result and RegWrite

Fig. 3 verifies the arithmetic and logic instructions. At PC = 8 (ADD) RD1 = 5 and RD2 = 10 give ALUResult = 15. At PC = 12 (SUB x4, x2, x1) the operands are 10 and 5 and the result is 5. The AND at PC = 16 gives 0, the OR and XOR at PCs 20 and 24 give 15, and the SLT at PC = 28 gives 1. The Result signal always follows the value to be written back, and RegWrite stays high except for two cycles: the SW at PC = 32 and the BEQ at PC = 44, which correctly do not write any register. For the JAL at PC = 56 the Result is 60 (PC + 4), and the final ADDI at PC = 64 gives 9.

6.3 Data Memory Interface

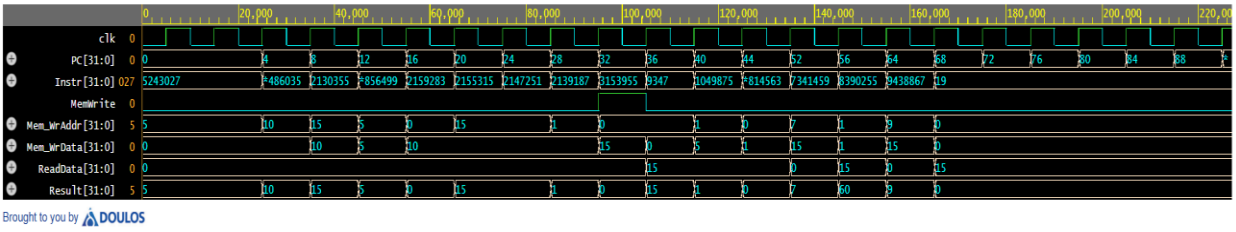


Fig. 4 Waveform of MemWrite, Mem_WrAddr, Mem_WrData, ReadData and Result

Fig. 4 shows the store and load operations. MemWrite is high for a single cycle only, during the SW instruction at PC = 32, with Mem_WrAddr = 0 and Mem_WrData = 15 (the value of x3). Once the value is written, ReadData at address 0 shows 15. In the following cycle (LW at PC = 36) the Result signal equals 15, which shows that the loaded data is selected for write-back and stored into x9.

6.4 Branch Logic

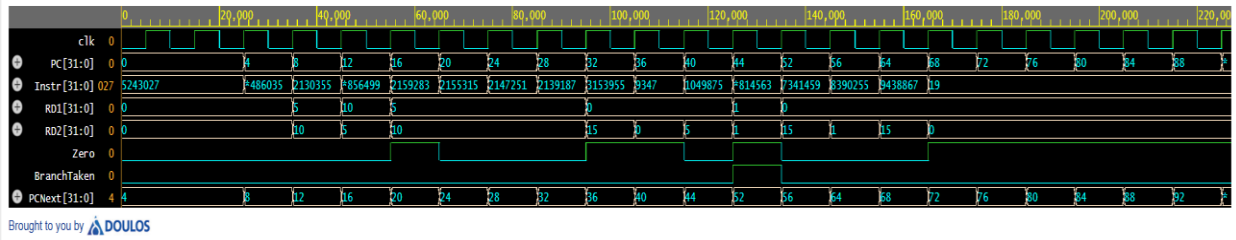


Fig. 5 Waveform of RD1, RD2, Zero, BranchTaken and PCNext

Fig. 5 shows that BranchTaken goes high only during the BEQ at PC = 44, where both RD1 and RD2 equal 1, and PCNext becomes 52. The Zero flag also goes high in other cycles, for example for the AND result of 0 or for a zero memory address, but it is not used for branch decisions because the design uses a dedicated comparator. It therefore does not cause false branches.

6.5 Destination Register and RegWrite

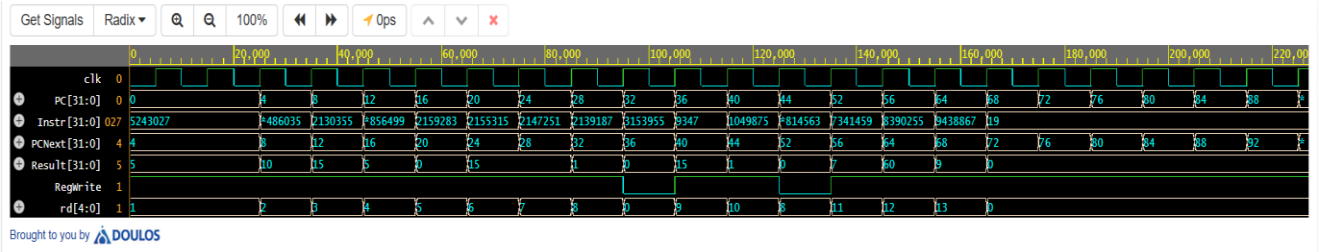


Fig. 6 Waveform of PCNext, Result, RegWrite and rd

Fig. 6 confirms the destination register for each instruction: rd increases from 1 to 8 for the first eight instructions, then rd = 9 for LW, 10 for ADDI x10, 11 for ADDI x11 (PC = 52), 12 for JAL and 13 for the last ADDI. During SW (PC = 32) and BEQ (PC = 44) the rd bits of the instruction word carry immediate bits (0 and 8), but RegWrite is low, so nothing is written. This is also why register x8 keeps its value 1 in the final checks.

6.6 Self-Checking Testbench Results

After 21 clock cycles the testbench compared the final processor state with the expected values. All checks passed, as shown below, and the testbench reported ALL TESTS PASSED.

Table 5 Register and memory check results

Location	Expected	Observed	Status
x0	0	0	PASS
x1	5	5	PASS
x2	10	10	PASS
x3	15	15	PASS
x4	5	5	PASS
x5	0	0	PASS
x6	15	15	PASS
x7	15	15	PASS
x8	1	1	PASS
x9	15	15	PASS
x10	1	1	PASS
x11	7	7	PASS
x12	60	60	PASS
x13	9	9	PASS
Data memory word 0	15	15	PASS

Result: 15 of 15 checks passed, 0 errors. The skipped instructions (ADDI x11, 99 and ADDI x13, 88) did not alter their destination registers, which proves that the taken branch and the JAL work correctly.

7. Observations and Limitations

- Each instruction completes in one clock cycle (CPI = 1). The clock period is limited by the longest path (instruction fetch, register read, ALU, data memory and write-back), so a pipelined design would give a higher clock frequency.
- Only word-sized memory access (LW and SW) is implemented. LB, LH, LBU, LHU, SB and SH are not supported.
- The data memory has 64 words and uses address bits [7:2], so addresses above 255 wrap around. Stores to non-word-aligned addresses are ignored.
- The test program exercises ADDI, ADD, SUB, AND, OR, XOR, SLT, SW, LW, BEQ and JAL. The remaining implemented instructions (shifts, SLTU, other branches, JALR, LUI, AUIPC) are present in the design but are not yet covered by the directed test.
- The testbench checks the final architectural state, not every intermediate cycle, so the waveforms were used as additional evidence of cycle-level behaviour.
- System instructions (ECALL, EBREAK, FENCE) and CSR instructions are not implemented.

8. Conclusion

A 32-bit single-cycle RISC-V processor supporting arithmetic, logic, load/store, branch and jump instructions was successfully designed in Verilog and verified by simulation. The self-checking testbench reported that all 14 register checks and the memory check passed. The waveforms confirm correct PC sequencing, ALU operation, register-file write control, data-memory access, branch decision and jump behaviour. The modular structure makes the design easy to read, test and extend.

9. Future Scope

- Extend the test program to cover all implemented instructions: shifts, SLTU, BNE/BLT/BGE/BLTU/BGEU, JALR, LUI and AUIPC, including corner cases such as negative numbers and writes to x0.
- Add byte and halfword loads and stores with a byte-enable memory interface.
- Convert the design into a 5-stage pipeline (IF, ID, EX, MEM, WB) with hazard detection and forwarding.
- Add a control unit with a separate main decoder and ALU decoder, and a cleaner control-signal interface.
- Synthesize the design on an FPGA and report area, timing and power.